

You are given an array of integers `heights` where `heights[i]` represents the height of a bar. The width of each bar is 1.

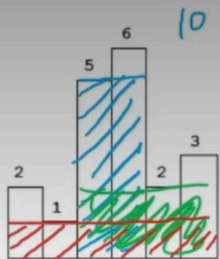
Return the area of the largest rectangle that can be formed among the bars.

Note: This chart is known as a **histogram**.

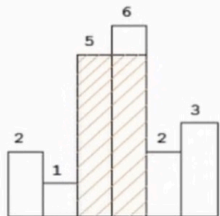
#### 84. Largest Rectangle in Histogram

Hard 3356 78 Add to List Share

Given  $n$  non-negative integers representing the histogram's bar height where the width of each bar is 1, find the area of largest rectangle in the histogram.



Above is a histogram where width of each bar is 1, given height = [2, 1, 5, 6, 2, 3].

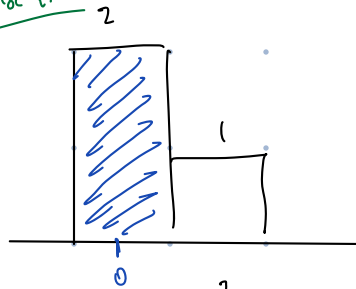


The largest rectangle is shown in the shaded area, which has area = 10 unit.

Example:

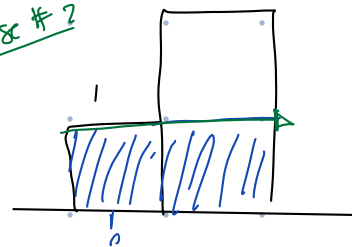
Input: [2, 1, 5, 6, 2, 3]  
Output: 10

Case #1



From the right side the max height is 2 & width is 1 in order for this to produce max-area.

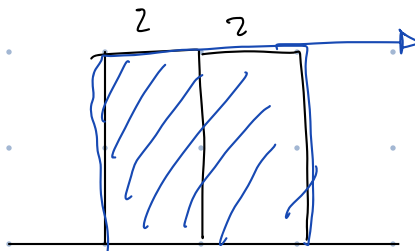
Case #2



From the right max height is 1 & width is 2.

Need to go both direction.

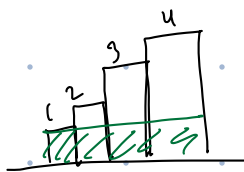
Case #3: even heights.



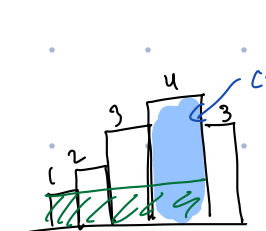
No "hole", so extend further to right.

Comment heights are increasing order, they will be popped.  
b/c in case #1, 2 cannot be extended further.

Case #4: increasing order.



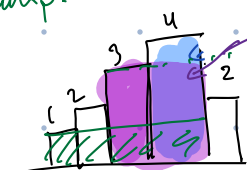
It can be extended further, but  
say you have a smaller right after.



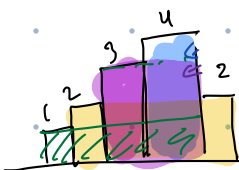
compute area at 4 & then pop it.

remove from  
consideration.

Examp: what if we have a 2 instead of a 3.



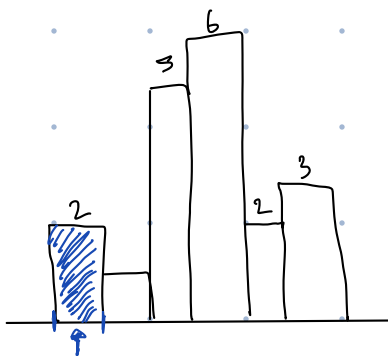
compute area & then pop it.



extending is possible.

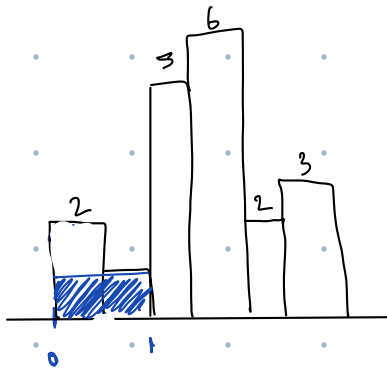
We only pop elements that are most recent. that is from  
the top; hence, we can use a stack.

Example. — Run through



| stack |        |
|-------|--------|
| index | height |
| 0     | 2      |

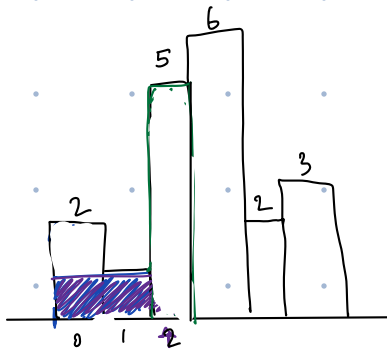
| max Area. |
|-----------|
| 2         |



| stack        |              |
|--------------|--------------|
| index        | height       |
| <del>0</del> | <del>2</del> |
| 0            | 1            |

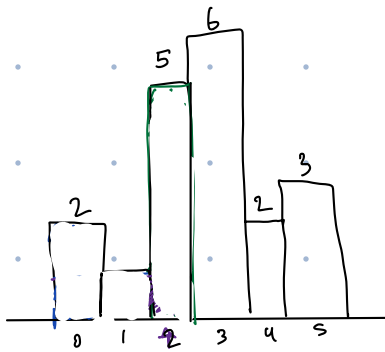
← pop b/c it is greater than current height.

max Area.  
2 ← k update max height is still 2:



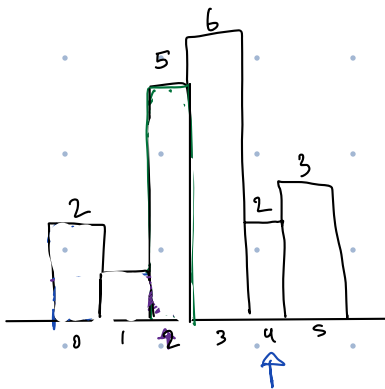
| stack        |              |
|--------------|--------------|
| index        | height       |
| <del>0</del> | <del>2</del> |
| 0            | 1            |
| 2            | 5            |

max Area.  
2



| stack        |              |
|--------------|--------------|
| index        | height       |
| <del>0</del> | <del>2</del> |
| 0            | 1            |
| 2            | 5            |
| 3            | 6            |

max Area.  
2



| stack        |              |
|--------------|--------------|
| index        | height       |
| <del>0</del> | <del>2</del> |
| 0            | 1            |
| 2            | 5            |
| 3            | 6            |
|              | 2            |

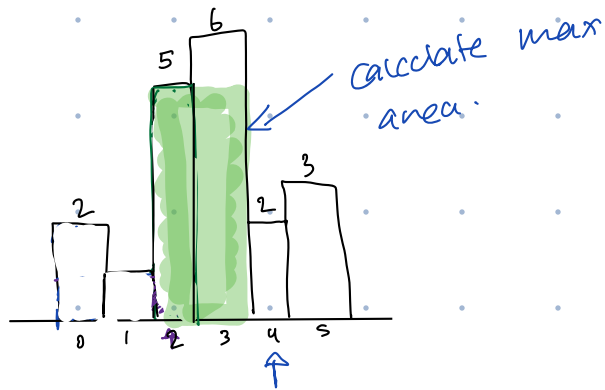
max Area.  
2

Since 2 is smaller than 6, we have to pop 6.

Before doing so, we need to calculate the area that the area of 6 could have created.

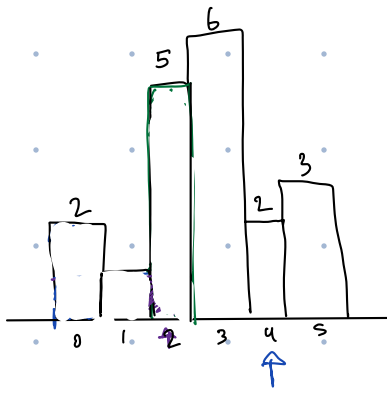
| Stack        |              | max Area.    |
|--------------|--------------|--------------|
| index        | height       |              |
| <del>0</del> | <del>2</del> | <del>2</del> |
|              |              | 6            |
| 0            | 1            |              |
| 2            | 5            |              |
| <del>3</del> | <del>6</del> |              |
|              | 2            |              |

Great, but now  $5 > 2$ , that means we have to also pop it.  
 It starts at 2 & stops at 4 (and the corners are at).



| Stack        |              | max Area.    |
|--------------|--------------|--------------|
| index        | height       |              |
| <del>0</del> | <del>2</del> | <del>2</del> |
|              |              | <del>6</del> |
| 0            | 1            | 10           |
| <del>2</del> | <del>5</del> |              |
| <del>3</del> | <del>6</del> |              |
|              | 2            |              |

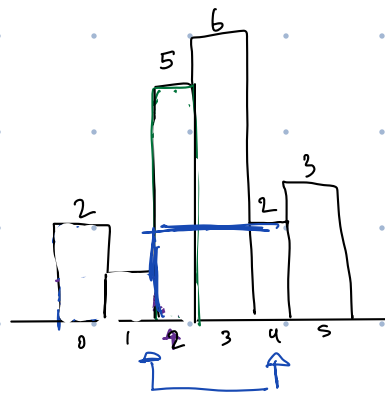
Now look at the top of the stack. It is 1, since that is smaller than 2, no need to pop it. The 1 can continue to be extended.



| Stack        |              |
|--------------|--------------|
| index        | height       |
| <del>0</del> | <del>2</del> |
| 0            | 1            |
| <del>2</del> | <del>5</del> |
| <del>3</del> | <del>6</del> |
|              | 2            |

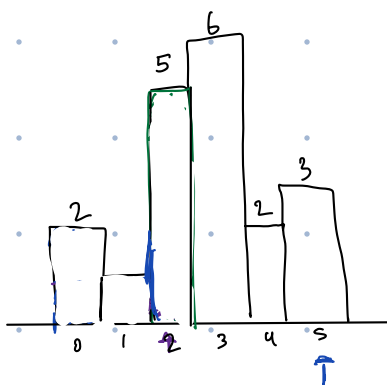
max Area.  
~~2~~  
~~6~~  
 10

The index of height 2, can be 4 but it can start at 1 b/c we popped two elements, meaning that it can be extended.



| Stack        |              |
|--------------|--------------|
| index        | height       |
| <del>0</del> | <del>2</del> |
| 0            | 1            |
| <del>2</del> | <del>5</del> |
| <del>3</del> | <del>6</del> |
| 2            | 2            |

max Area.  
~~2~~  
~~6~~  
 10



| Stack        |              |
|--------------|--------------|
| index        | height       |
| <del>0</del> | <del>2</del> |
| 0            | 1            |
| <del>2</del> | <del>5</del> |
| <del>3</del> | <del>6</del> |
| 2            | 2            |
| 5            | 3            |

max Area.  
~~2~~  
~~6~~  
 10

$3 > 2$ , so we do not need to pop it. B/c we cannot extend backwards anymore, then the start index is 5.

Now, we have 3 elements still in stack, meaning we were able to extend all the way to end of histogram.

Thus, we need to iterate these positions & calculate areas.

| Stack |        |
|-------|--------|
| index | height |
| 0     | 1      |
| 2     | 2      |
| 5     | 3      |

at index 5 it started & went all way to end, so length is 1.

area = 3.

$3 < 10$ , so no update.

| Stack |        |
|-------|--------|
| index | height |
| 0     | 1      |
| 2     | 2      |

at index 2 it started & went all way to end, so length is 4

$$\underline{\text{area} = 8}$$

$8 < 10$ , so no update.

| Stack |        |
|-------|--------|
| index | height |
| 0     | 1      |

at index 0 it started & went all way to end, so length is 6

$$\underline{\text{area} = 6}$$

$6 < 10$ , so no update.

Thus max area is 10.

We iterate elements once. Push elements once & pop once, meaning. the overall time complexity is  $O(n)$ .

Worst case stack space complexity is  $O(n)$ .

## Pseudo-code

maxArea = 0

stack = [ ] # pairs: (index, height)

for indx & curr-h in heights:

start = indx.

while stack not empty & the top of stack > curr-h:

① i, h = stack.pop()

② new-area = h \* (indx - i)

③ update max area.

④ extend our start index backwards to the index we just popped.

↳ start = i

# add pair to the stack.

stack.append( (start, curr-h) )

# Now take care of any entries remaining in stack

# These are entries that can be extended all the way

# to the end



for i, h in stack:

maxArea = max(maxArea,

$h * (\text{len}(\text{heights}) - i)$ )

  
the end  
of the  
histogram.

return maxArea.